Entertainment Problem Statement

SECTION A — SCHEMA ANALYSIS AND NORMALIZATION

Given Schema

Bookings (

  booking_id,

  user_name,

  user_email,

  movie_name,

  theatre_name,

  theatre_location,

  show_time,

  seat_numbers,

  total_price,

  payment_status

)

Tasks

A1. Identify Issues and list them with reasons

- Not Normalised, no partial or transitive dependency
- Data redundancy: In this type of schema, one particular user will book 10 times from the same Email or User_id, and the 10 times his User_id and Email will be saved
- Deletion Anomalies: If you delete the only booking for a specific movie because the customer cancelled, you lose all information about that movie (its name and existence) because the movie data only exists inside a booking record.
- Update Anomaly: If a theatre changes its name from "ABCD" to "WXYZ," you have to search through thousands of bookings to change the name in every single row. If you miss one, your data is wrong.
- Storing seat numbers as A1, A2, A3 in one box is a bad idea. The computer sees that as one long word. It becomes very hard to check if the A2 seat is specifically taken or empty.
- Redundant like User_name,Movie_name,theatre_name can be same in some cases
- Information like user_email, movie_name, and theatre_location is repeated every time a booking is made.

- Non-key attributes are dependent on other non-key attributes. For example, theatre_location depends on theatre_name, not the booking_id.

	user_email	movie_name	theatre_name	theatre_location	show_time	seat_numbers
1	Unaman@gmail.com	Interstellar	PVR Cinemas	Mangaluru	2026-05-10 18:30:00	A1, A2
2	UjwalS@gmail.com	Inception	Imax	Bangalore	2026-05-10 21:00:00	B5
3	Rai@gamil.com	Interstellar	PVR Cinemas	Mangaluru	2026-05-10 18:30:00	C10, C11
4	Preetham@gmail.com	The Batman	Cinepolis	Manipal	2026-05-11 14:00:00	F1
5	KST@gmail.com	The Batman	Cinepolis	Manipal	2026-05-11 14:00:00	F2

A2. Normalize the Schema

- 1NF

Since one user can book 2 or more seats finding the availability of seats it wont be as much easy comparatively. Thus we make use of 1NF, for which each seat number will be given in each row respectively.

	booking_id	user_name	user_email	movie_name	theatre_name	theatre_location	show_time	seat_numbe
1	101	Naman U	Unaman@gmail.com	Interstellar	PVR Cinemas	Mangaluru	2026-05-10 18:30:00	A1
2	101	Naman U	Unaman@gmail.com	Interstellar	PVR Cinemas	Mangaluru	2026-05-10 18:30:00	A2
3	102	Ujwal Shetty	UjwalS@gmail.com	Inception	Imax	Bangalore	2026-05-10 21:00:00	B5
4	103	Sohan P Rai	Rai@gamil.com	Interstellar	PVR Cinemas	Mangaluru	2026-05-10 18:30:00	C10
5	103	Sohan P Rai	Rai@gamil.com	Interstellar	PVR Cinemas	Mangaluru	2026-05-10 18:30:00	C11
6	104	Preetham	Preetham@gmail.com	The Batman	Cinepolis	Manipal	2026-05-11 14:00:00	F1
7	105	Nikhil KT	KST@gmail.com	The Batman	Cinepolis	Manipal	2026-05-11 14:00:00	F2

Note: PRIMARY KEY [PK]

FOREIGN KEY [FK]

Bookings Table:

Bookings (

booking_id [PK]

seat_numbers [PK]

user_name

user_email

movie_name

theatre_name

theatre_location

show_time

total_price

payment_status

);

- 2NF

To convert our Bookings table into 2NF, we remove partial dependency (seat_numbers depends only on booking). We split the Users table so that user details are stored only once and can be linked to multiple bookings.We created separate Users, Movies, Theatres, Shows, Bookings, and Booking_Seats tables to remove repeated and redundant data in 2NF

1. Users Table

```
Users (
  user_id [PK],
  user_name,
  user_email
)
```

2. Movies Table

```
Movies (
  movie_id [PK],
  movie_name
)
```

3. Theatres Table

```
Theatres (
  theatre_id [PK],
  theatre_name,
  theatre_location
)
```

4. Shows Table

```
Shows (
  show_id [PK],
  movie_id [FK ],
  theatre_id [FK],
  show_time
)
```

5. Bookings Table

```
Bookings (
  booking_id [PK],
  user_id [FK],
  show_id [FK],
  total_price,
  payment_status
)
```

6. Booking_Seats Table

```
Booking_Seats (
  booking_id [FK],
  seat_number [PK],
)
```

- 3NF

In 3NF, we split the tables to remove transitive dependencies, so every non-key attribute depends only on the primary key. Movie and theatre details are kept separately, which with the help of Foreign Key, are referred. While user details are stored separately in the Users table.

1. Users Table

```
Users (
  user_id [PK],
  name,
  email
)
```

2. Movies Table

```
Movies (
  movie_id [PK],
  movie_name
)
```

3. Theatres Table

```
Theatres (
  theatre_id [PK],
  theatre_name,
  theatre_location
)
```

4. Screens Table

```
 Screens (
  screen_id [PK],
  theatre_id [FK],
  screen_name
)
```

5. Shows Table

```
Shows (
  show_id [PK],
  movie_id [FK],
```

  screen_id [FK],
  show_time
)

6. Bookings Table

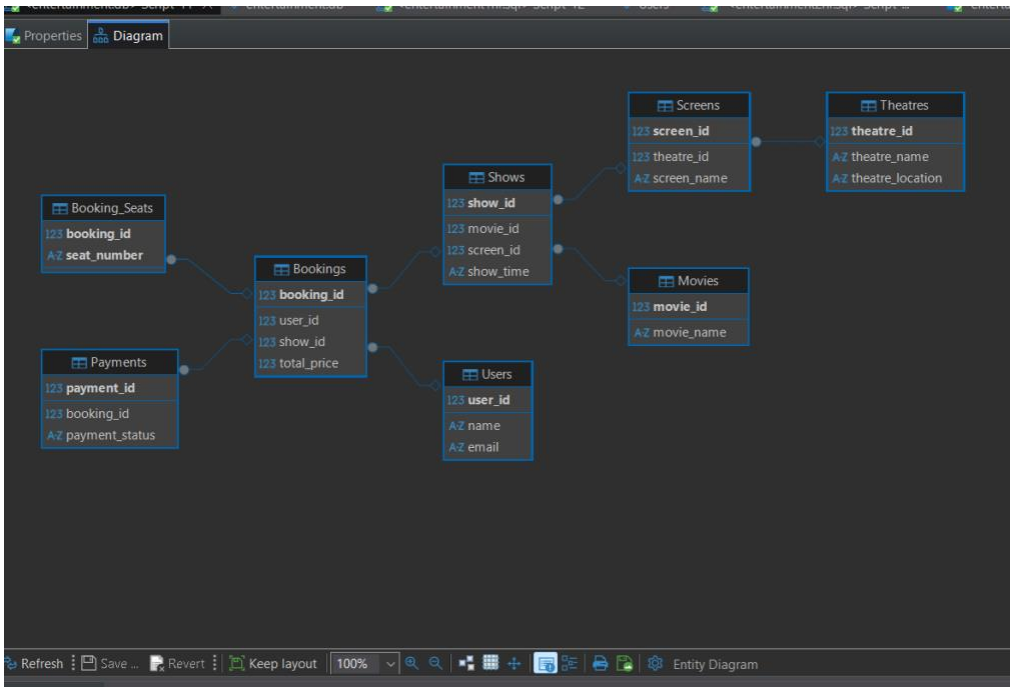
Bookings (
  booking_id [PK],
  user_id [FK],
  show_id [FK],
  total_price
)

7. Payments Table

Payments (
  payment_id [PK],
  booking_id [FK],
  payment_status
)

8. Booking_Seats Table

Booking_Seats (
  booking_id [FK],
  seat_number,
)



3NF is achieved by further refining the schema so that each table stores only its own attributes, such as Users, Movies, Theatres, Shows, Bookings, Seats, and Payments.

Non-key attributes (like movie name or theatre location) depend only on their primary keys and not on other non-key attributes, removing transitive dependencies.
All relationships (bookings, users, seats, and payments) are handled using foreign keys, ensuring minimal redundancy and a clean, fully normalized design.

A3. Assumptions

- Each user is uniquely identified by user_id, and email is unique for every user.
- Each movie has a unique movie_id and is stored only once in the system.
- Each theatre has a unique theatre_id, and its location is stored as a single attribute.
- A show represents map of a specific movie screened at a specific theatre at a particular time.
- A booking can include multiple seats, handled through the Booking_Seats table.
- Each seat number is unique within a booking, enforced using a composite key.
- Payment details are stored separately, and each booking has at most one payment record.
- Payment Status depends only on payment_id, regardless of any other entities.

SECTION B — DATABASE DESIGN FROM REQUIREMENTS

Tasks

B1. Requirement Clarification

Should a user be able to book seats across multiple shows in a single booking?

How should seat availability be handled in real time?

What types of seat categories are supported?

How is pricing determined?

Should users be able to cancel or modify bookings?

Do we need to track show status ?

Should admins be able to add/update movies, theatres, shows, and pricing?

What payment methods need to be supported?

Do we need to support partial cancellations or modifications?

B2. Database Schema Design (Normalized)

1. **Location & Theatre Management**
   - **Cities**

Cities (
  city_id [PK],
  city_name,
)

   - **Theatres**

Theatres (
  theatre_id [PK],
  Theatres_name,
  Theatres_location
  city_id [FK]
)

   - **Screens**

Screens (
  screen_id [PK],
  theatre_id [FK],
  screen_number,

)

2. **Movie & Show Management**

   - **Movies**

Movies (
  movie_id [PK],
  movie_name,
  description
)

   - **Genres**

Genres (
  genre_id [PK],
  genre_name
)

   - **Movie_Genres**

Movie_Genres (
  movie_id [FK],
  genre_id [FK],
)

- **Shows**

```
Shows (
  show_id [PK],
  movie_id [FK],
  screen_id [FK],
  date
)
```

3. Seats & Pricing

- **Seats**

```
Seats (
  seat_id [PK],
  screen_id [FK],
  seat_number,
)
```

- **Show_Seats**

```
Show_Seats (
  show_seat_id [PK],
  show_id [FK],
  seat_id [FK],
)
```

4. Users & Bookings

- **Users**

```
Users (
  user_id [PK],
  name,
  email
)
```

- **Bookings**

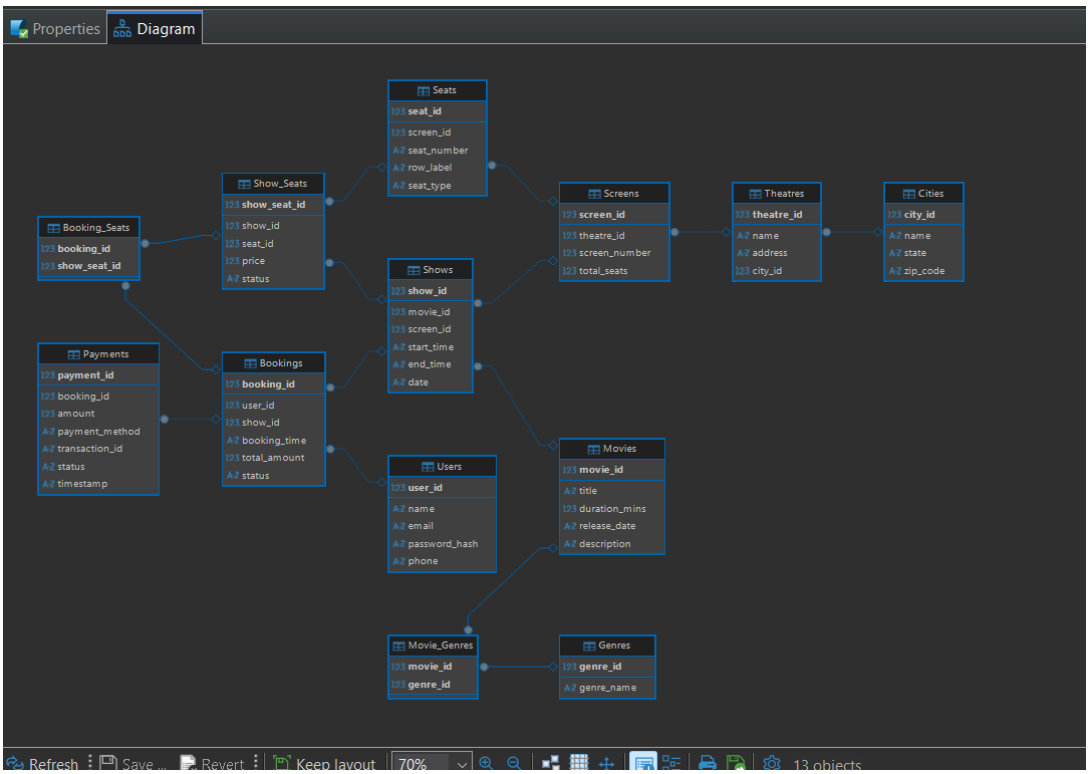
```
Bookings (
  booking_id [PK],
  user_id [FK],
  show_id [FK],
  booking_time,
  total_amount,
  status
)
```

- **Booking_Seats**

Booking_Seats (
  booking_id [FK],
  show_seat_id [FK],
)

- **Payments**

Payments (
  payment_id [PK],
  booking_id [FK],
  status,
  timestamp
)



B3. Normalization

The above database schema is fully normalized up to **Third Normal Form (3NF)**. It eliminates data redundancy by separating entities into distinct tables such as Cities, Theatres, Movies, Shows, Users, Bookings, Seats, and Payments. All non-key attributes depend only on their respective primary keys, and transitive dependencies have been removed using proper foreign key relationships as we can in same as that of Section A2.

First Normal Form (1NF) : All tables are designed so that each column holds a single, indivisible value. For example, seat details are not stored as a list in one field; instead, each seat is recorded separately. This ensures the data is clean and easy to query.

Second Normal Form (2NF): All non-key attributes fully depend on the primary key. In cases where composite keys are used (like in mapping tables), no attribute depends on just part of the key. This removes partial dependency and avoids unnecessary duplication.

Third Normal Form (3NF): There are no indirect dependencies between attributes. Every non-key attribute depends only on the primary key and not on another non-key attribute. For example, theatre details are stored separately rather than being repeated in multiple places.

Update During Development

We removed user_id from the Bookings table and introduced a new Booking_Users table to support multiple users per booking.
We also updated Booking_Seats to include user_id, so each seat is linked to a specific user within the booking.

```
Bookings (
  booking_id [PK],
  show_id [FK],
  booking_time,
  total_amount,
  status
)
```

Booking_Users

```
Booking_Users (
  booking_id [ FK],
  user_id [ FK]
)
```

Booking_Seats

```
Booking_Seats (
  booking_id [PK],
  user_id [PK],
  show_seat_id [FK]
)
```

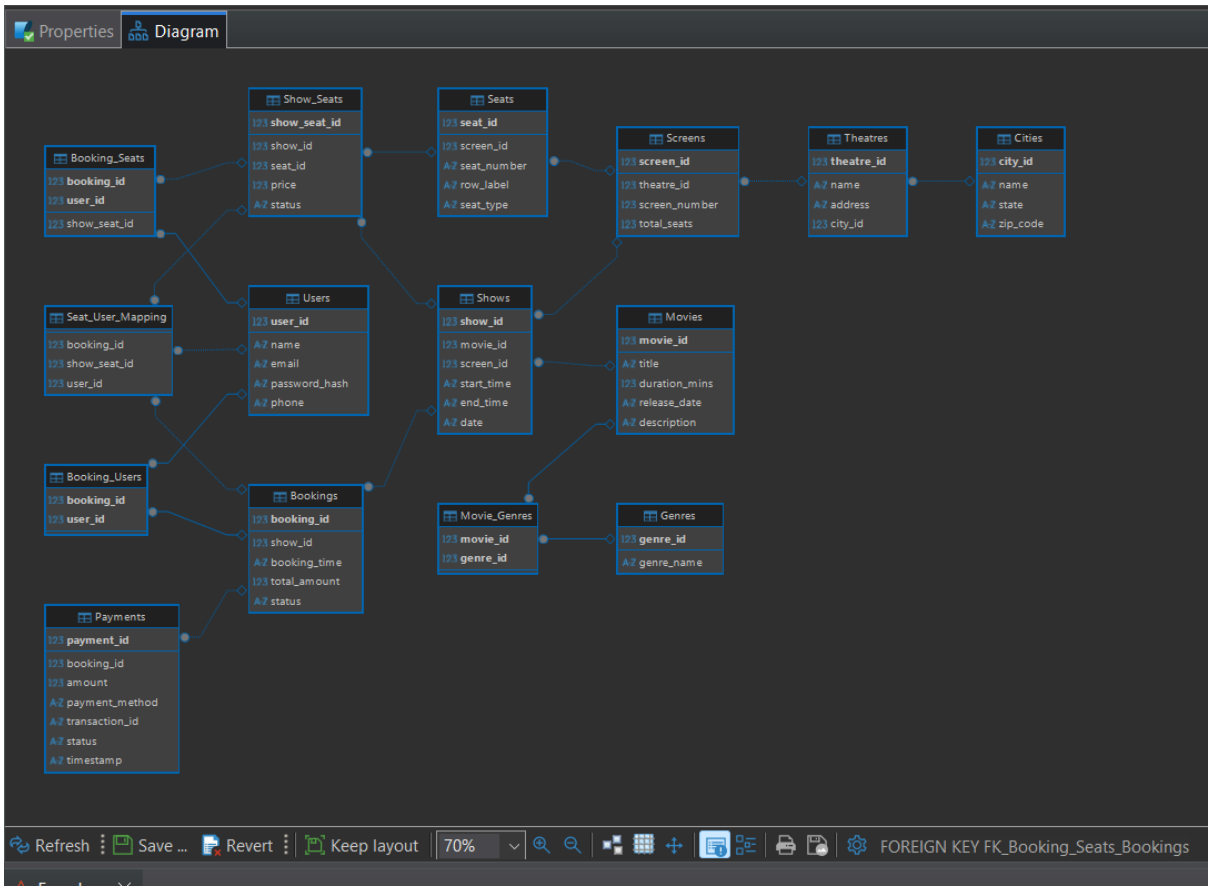
```
Seat_User_Mapping (

  booking_id [FK],

  show_seat_id  [FK],

  user_id [FK]

)
```



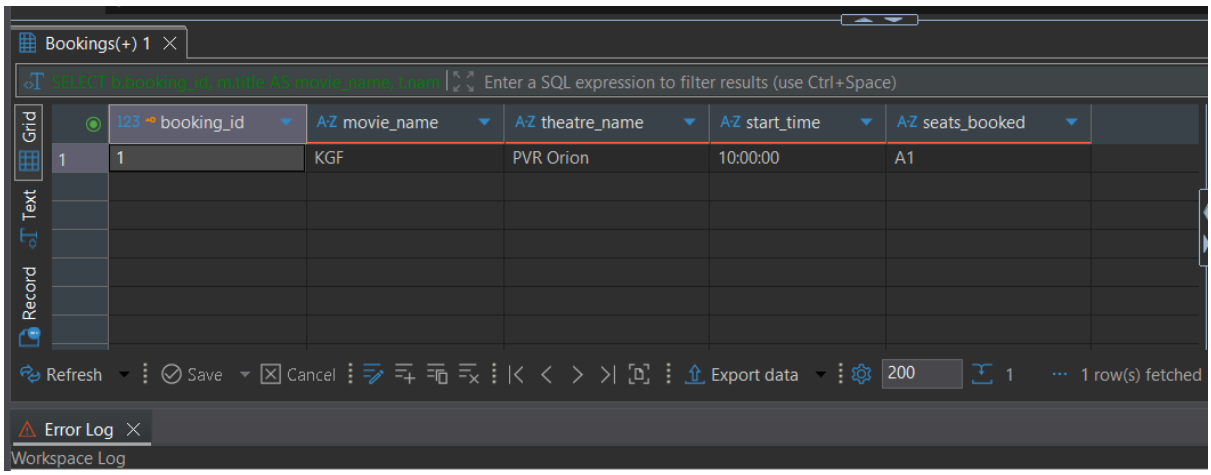
SECTION C — QUERIES AND TRANSACTIONS

1. Retrieve User Bookings in a Date Range

This query joins several tables to translate IDs into human-readable names and aggregates multiple seats into a single string for better readability.

```
SELECT
    b.booking_id,
    m.title AS movie_name,
    t.name AS theatre_name,
    s.start_time,
    GROUP_CONCAT(st.seat_number, ', ') AS seats_booked
FROM Bookings b
JOIN Shows s ON b.show_id = s.show_id
JOIN Movies m ON s.movie_id = m.movie_id
```

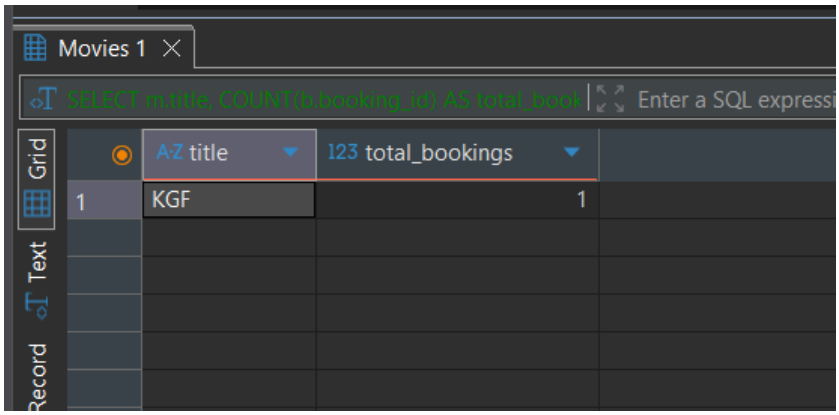
```
JOIN Screens sc ON s.screen_id = sc.screen_id
JOIN Theatres t ON sc.theatre_id = t.theatre_id
JOIN Booking_Users bu ON b.booking_id = bu.booking_id
JOIN Booking_Seats bs ON b.booking_id = bs.booking_id AND bu.user_id = bs.user_id
JOIN Show_Seats ss ON bs.show_seat_id = ss.show_seat_id
JOIN Seats st ON ss.seat_id = st.seat_id
WHERE bu.user_id = 1
  AND s.date BETWEEN '2026-04-01' AND '2026-05-30'
GROUP BY b.booking_id, m.title, t.name, s.start_time;
```



2. Most Frequently Booked Movie

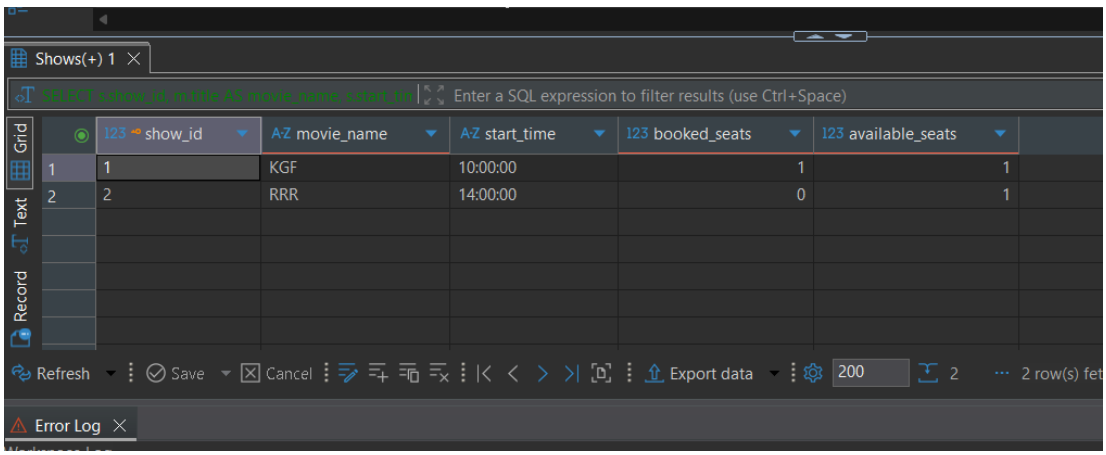
This utilizes COUNT and GROUP BY to aggregate the total number of bookings per movie.

```
SELECT
    m.title,
    COUNT(b.booking_id) AS total_bookings
FROM Movies m
JOIN Shows s ON m.movie_id = s.movie_id
JOIN Bookings b ON s.show_id = b.show_id
GROUP BY m.movie_id, m.title
ORDER BY total_bookings DESC
LIMIT 1;
```



3. Show Availability per Theatre and Date

This query calculates availability by taking the static total_seats from the Screens table and subtracting the count of seats marked as 'Booked' in the Show_Seats table for that specific show.



SELECT

s.show_id,

m.title **AS** *movie_name*,

s.start_time,

COUNT(CASE WHEN *ss*.status = 'Booked' **THEN** 1 **END) AS** *booked_seats*,

COUNT(CASE WHEN *ss*.status = 'Available' **THEN** 1 **END) AS** *available_seats*

FROM Shows *s*

JOIN Movies *m* **ON** *s*.movie_id = *m*.movie_id

JOIN Screens *sc* **ON** *s*.screen_id = *sc*.screen_id

JOIN Theatres *t* **ON** *sc*.theatre_id = *t*.theatre_id

JOIN Show_Seats *ss* **ON** *s*.show_id = *ss*.show_id

WHERE *t*.theatre_id = 1

AND *s*.date = '2026-05-01'

GROUP BY *s*.show_id, *m*.title, *s*.start_time;

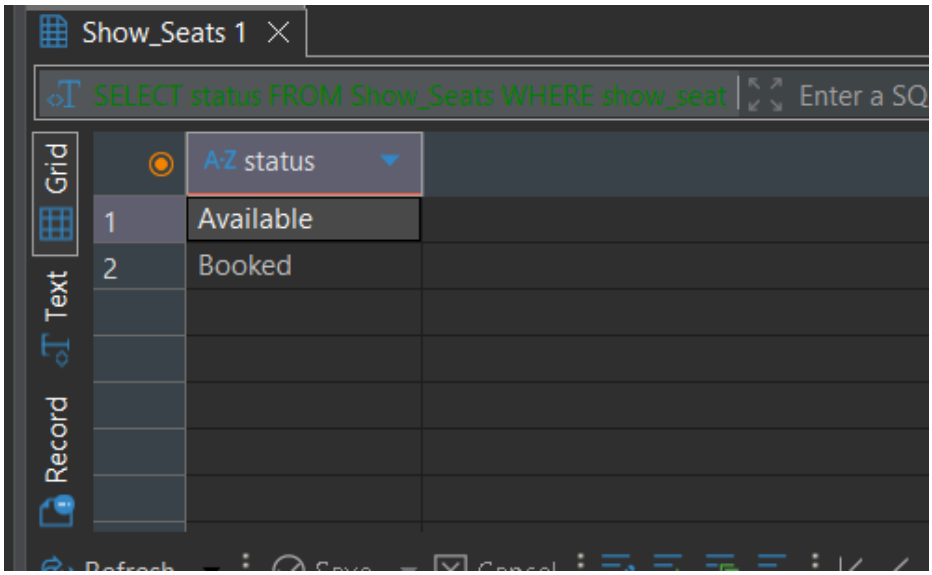
4. Booking Transaction with Safety Checks

- Begin the transaction

BEGIN TRANSACTION;

1. Check availability (optional view step)

SELECT status
FROM Show_Seats
WHERE show_seat_id IN (1, 2);



2. Insert booking
INSERT INTO Bookings (booking_id, show_id, booking_time, total_amount, status)
VALUES (4, 1, CURRENT_TIMESTAMP, 300.00, 'Confirmed');

	booking_id	show_id	booking_time	total_amount	status
1	1	1	2026-05-01 09:00:00	500	Confirmed
2	2	2	2026-05-01 12:00:00	200	Pending
3	3	3	2026-05-01 17:00:00	300	Confirmed
4	4	1	2026-04-29 06:11:47	300	Pending

Unique key: PRIMARY KEY booking_id

3. Insert user-seat mapping

```
INSERT INTO Booking_Seats (booking_id, user_id, show_seat_id)
VALUES
(2, 2, 1),
(2, 2, 2);
```

	booking_id	user_id	show_seat_id
1	2	2	1
2	2	2	2

4. Try to reserve seats ONLY if available

```
UPDATE Show_Seats
SET status = 'Booked'
WHERE show_seat_id IN (1, 2)
AND status = 'Available';
```

Properties | Data | Diagram

Enter a SQL expression to filter results (use Ctrl+Space)

	show_seat_id	show_id	seat_id	price	status
1	1	1	1	250	Booked
2	2	1	2	250	Booked
3	3	2	3	200	Available

5. Commit if everything is fine

```
COMMIT;
```